

Compiladores: Panorama General y Contexto

Cuestionario Diagnóstico Autómatas

DRA. MELISSA BRIGITTHE HINOJOSA CABELLO

Universidad Nacional Autónoma de México

Facultad de Ingeniería

1ro. de Septiembre de 2026





1. Una expresión regular puede describir completamente la sintaxis de un lenguaje de programación.

Verdadero

Falso

→ Las expresiones regulares solo describen lenguajes regulares, insuficientes para estructuras anidadas.

2. Un autómata finito puede reconocer estructuras con anidamiento arbitrario.

Verdadero

Falso

→ Los autómatas finitos no tienen memoria para manejar anidamiento.



¿Qué es una expresión regular?

Expresión regular

Es una representación algebraica de un lenguaje regular que puede ser reconocido por un autómata finito.

Las expresiones regulares describen lenguajes regulares, es decir, patrones lineales sin estructura jerárquica. Un lenguaje de programación incluye:

- Bloques.
- Paréntesis anidados.
- Estructuras de control.

→ Los lenguajes de programación requieren memoria para contar niveles de anidamiento, algo que las expresiones regulares no pueden hacer.



¿Qué es un autómata finito?

Autómata finito

Es una máquina abstracta con un número finito de estados que reconoce lenguajes regulares leyendo la entrada símbolo por símbolo.

→ Un autómata finito tiene estados pero no tiene memoria adicional, por lo que no puede contar indefinidamente. Reconocer estructuras como $((()))$ implica '*recordar*' cuántos paréntesis se han abierto.



3. Toda gramática libre de contexto puede representarse mediante expresiones regulares.

Verdadero

Falso

→ Las gramáticas libres de contexto describen lenguajes más poderosos que los regulares.

4. Un lenguaje de programación típico requiere al menos una gramática libre de contexto para describir su sintaxis.

Verdadero

Falso

→ Las estructuras jerárquicas requieren gramáticas libres de contexto.



¿Qué es una gramática?

→ Es un conjunto de reglas que describe cómo se pueden construir cadenas válidas en un lenguaje.

¿Qué es una gramática libre de contexto?

Gramática libre de contexto

Es un conjunto de reglas que generan cadenas de un lenguaje cuya estructura puede reconocerse mediante un autómata con pila y pueden describir paréntesis balanceados, expresiones aritméticas, y estructuras anidadas.

→ Las expresiones regulares son más limitadas y no pueden describir ninguno de estos tres elementos.



5. Todo lenguaje regular es también un lenguaje libre de contexto.

Verdadero

Falso

→ Los lenguajes regulares son un subconjunto de los libres de contexto, por lo que todo lenguaje regular puede describirse también como libre de contexto, pero no al revés.

6. Las expresiones regulares pueden describir correctamente paréntesis balanceados.

Verdadero

Falso

→ Los paréntesis balanceados requieren memoria (pila) y las expresiones regulares no tienen memoria.



Lenguaje regular

Un lenguaje regular es un conjunto de cadenas que puede ser descrito mediante una expresión regular y reconocido por un autómata finito.

→ En Compiladores, los lenguajes regulares se utilizan para definir los tokens durante el análisis léxico.

Lenguaje libre de contexto

Un lenguaje libre de contexto es un conjunto de cadenas que puede ser generado por una gramática libre de contexto y reconocido por un autómata con pila.

→ En Compiladores, los lenguajes libres de contexto se utilizan para definir la estructura sintáctica del lenguaje (expresiones, sentencias, bloques, etc.).



7. Una gramática ambigua puede generar más de un árbol de derivación para la misma cadena.

Verdadero

Falso

→ Esa es la definición de ambigüedad.

8. Todo autómata finito no determinista puede convertirse en uno determinista equivalente.

Verdadero

Falso

→ Existe el algoritmo de subconjuntos AFN $\rightarrow$ AFD.

9. La gramática de un lenguaje define el significado del programa.

Verdadero

Falso

→ La gramática define la sintaxis, no la semántica.



Autómata finito no determinista

Un AFN es un autómata finito que puede tener múltiples transiciones para el mismo símbolo y decisiones no deterministas, o bien, incluir transiciones ε (movimientos sin consumir un símbolo).

Autómata finito determinista

Un AFD es un autómata finito donde cada símbolo determina de forma única el siguiente estado; es decir, que para cada estado y símbolo existe exactamente una transición, por lo que no existen transiciones ε .

→ En Compiladores, se convierten automáticamente expresiones regulares en AFN y luego se convierte en AFD para implementarlo eficientemente en el analizador léxico.



10. Dada la gramática:

$$E \rightarrow E + T \mid T$$
$$T \rightarrow \text{num}$$

¿Qué tipo de gramática es?

- Regular
- **Libre de contexto**
- Sensible al contexto
- Irrestringida

→ Una gramática libre de contexto tiene la forma $A \rightarrow \alpha$, donde A es un único no terminal y α puede ser cualquier combinación de terminales y no terminales.

11. Da un ejemplo de una cadena válida generada por la gramática de la pregunta anterior.

- num
- num + num
- num + num + num

→ Las producciones permiten concatenar sumas porque E se puede expandir recursivamente en otra suma.



12. Dada la gramática:

$S \rightarrow a S b \mid \varepsilon$

¿Qué tipo de gramática es?

- Regular
- **Libre de contexto**
- Sensible al contexto
- Irrestricta

→ No puede expresarse con expresiones regulares debido al anidamiento implícito.

13. Escribe una producción en BNF para una instrucción *if* simple.

■ `<if> ::= if ( <cond> ) <stmt>`



Backus–Naur Form

Es una notación formal utilizada para describir la sintaxis de lenguajes de programación mediante reglas gramaticales y sirve como base para construir el analizador sintáctico.

Una producción BNF es una regla de la forma:

$$\langle \text{no_terminal} \rangle ::= \langle \text{expresion} \rangle$$

Donde $\langle \text{no_terminal} \rangle$ es un símbolo que puede expandirse y $\langle \text{expresion} \rangle$ puede contener terminales y no terminales.



Tipo/Nombre	Producción BNF
Programa	$\langle \text{programa} \rangle ::= \langle \text{lista_sentencias} \rangle$
Lista de sentencias	$\langle \text{lista_sentencias} \rangle ::= \langle \text{sentencia} \rangle \mid \langle \text{sentencia} \rangle \langle \text{lista_sentencias} \rangle$
Asignación	$\langle \text{sentencia} \rangle ::= \langle \text{identificador} \rangle = \langle \text{expresion} \rangle ;$
Condicional	$\langle \text{sentencia} \rangle ::= \text{if} (\langle \text{expresion} \rangle) \langle \text{bloque} \rangle$ $\langle \text{sentencia} \rangle ::= \text{if} (\langle \text{expresion} \rangle) \langle \text{bloque} \rangle \text{ else } \langle \text{bloque} \rangle$
Ciclo while	$\langle \text{sentencia} \rangle ::= \text{while} (\langle \text{expresion} \rangle) \langle \text{bloque} \rangle$
Bloques	$\langle \text{bloque} \rangle ::= \{ \langle \text{lista_sentencias} \rangle \}$
Expresión	$\langle \text{expresion} \rangle ::= \langle \text{expresion} \rangle \pm \langle \text{termino} \rangle \mid \langle \text{termino} \rangle$
Término	$\langle \text{termino} \rangle ::= \langle \text{termino} \rangle * \langle \text{factor} \rangle \mid \langle \text{factor} \rangle$
Factor	$\langle \text{factor} \rangle ::= (\langle \text{expresion} \rangle) \mid \langle \text{identificador} \rangle \mid \langle \text{numero} \rangle$
Identificador	$\langle \text{identificador} \rangle ::= \text{id}$
Número	$\langle \text{numero} \rangle ::= \text{num}$

14. Escribe una producción en BNF para una asignación simple $x = \text{expresión}$;

■ $\langle \text{assign} \rangle ::= \langle \text{id} \rangle = \langle \text{expr} \rangle ;$

15. Dada la gramática:

$S \rightarrow a S \mid a$

¿Qué lenguaje genera?

■ Una o más letras 'a' (a^+)

→ Cada derivación añade una 'a' y termina en 'a'.

16. ¿Qué problema tiene la siguiente gramática?

$E \rightarrow E + E \mid E * E \mid \text{num}$

- No genera expresiones válidas.
- No es libre de contexto.
- **Es ambigua.**
- No puede implementarse.

→ No define precedencia ni asociatividad.



17. En la expresión $3 + 4 * 5$, ¿qué operación se evalúa primero?

- $3 + 4$
- $4 * 5$
- Ambas al mismo tiempo.

→ La multiplicación tiene mayor precedencia.

18. Explica brevemente cómo un árbol de derivación refleja la precedencia de operadores.

- Operadores de mayor precedencia aparecen más profundos.
- El árbol refleja jerarquía.



19. ¿Por qué estas dos expresiones tienen árboles distintos?

$$(3 + 4) * 5$$

$$3 + (4 * 5)$$

- Los paréntesis cambian la estructura.
- Cambia el orden de evaluación.

20. ¿Cuál árbol corresponde a $a + b * c$?

- **Árbol con $+$ como raíz.**
- **Árbol con $*$ más profundo.**

21. ¿Siempre es posible eliminar paréntesis sin cambiar el significado? ¿Por qué?

- No siempre.
- Depende de precedencia y asociatividad.



22. ¿Por qué un compilador no puede basarse únicamente en expresiones regulares?

- A) **Porque las expresiones regulares no pueden manejar estructuras anidadas.**
- B) **Porque no pueden describir dependencias jerárquicas.**
- C) Porque no permiten reconocer ningún tipo de operador.
- D) **Porque no pueden representar la sintaxis completa de un lenguaje de programación.**
- E) Porque no permiten definir palabras reservadas.



23. ¿Qué tipo de errores detecta el análisis léxico que el análisis sintáctico normalmente no detecta?

- A) **Uso de símbolos inválidos en el alfabeto del lenguaje.**
- B) **Tokens mal formados (por ejemplo, un número con formato incorrecto).**
- C) Falta de punto y coma al final de una instrucción.
- D) **Comentarios que no se cierran correctamente.**
- E) Asignación entre tipos incompatibles.



24. ¿Por qué el análisis léxico puede implementarse con autómatas finitos, pero el análisis sintáctico no?

- A) **Porque el análisis léxico reconoce lenguajes regulares.**
- B) **Porque el análisis sintáctico requiere manejar estructuras anidadas.**
- C) **Porque los autómatas finitos no tienen memoria para manejar anidamiento.**
- D) Porque el análisis sintáctico trabaja directamente con caracteres individuales.
- E) Porque los autómatas finitos no pueden reconocer ningún tipo de patrón repetitivo.



25. ¿Por qué una gramática ambigua representa un problema para un compilador?

- A) **Porque una misma cadena puede generar múltiples árboles sintácticos.**
- B) **Porque puede haber más de una interpretación estructural válida.**
- C) Porque impide que el lenguaje tenga operadores.
- D) **Porque dificulta la generación correcta de código.**
- E) **Porque el compilador no sabría qué significado elegir.**



26. ¿Por qué un compilador normalmente trabaja con un árbol sintáctico abstracto (AST) en lugar del árbol de derivación completo?

- A) **Porque el árbol de derivación contiene información redundante.**
- B) **Porque el AST elimina detalles puramente sintácticos.**
- C) **Porque el AST es más adecuado para análisis semántico y generación de código.**
- D) Porque el árbol de derivación no representa la estructura del programa.
- E) **Porque el AST es más simple y manejable computacionalmente.**



27. ¿Qué ventajas tiene dividir un compilador en fases independientes?

- A) **Facilita la modularidad del sistema.**
- B) **Mejora el mantenimiento y la extensión del compilador.**
- C) **Permite reutilizar componentes.**
- D) Reduce completamente la complejidad del compilador.
- E) **Hace más claro el diseño del sistema.**